Course Title: Cryptography and Network Security Lab	**Course Code**: BCSE309P
Faculty: Prof. Karthika V	Slot: *L15+L16*
Regno: *23BCE1099*	Name: *Arkita Barua*

Ex5

ELGAMAL CRYPTOGRAPHY

Code

Sender

```
import java.io.*;
import java.math.BigInteger;
import java.net.Socket;
import java.security.SecureRandom;
import java.util.Scanner;

public class elgamal_client_sender {

    public static void main(String[] args) {

        try (Socket s = new Socket("localhost", 5000)) {

            DataInputStream dis = new DataInputStream(s.getInputStream());
            DataOutputStream dos = new DataOutputStream(s.getOutputStream());

            // Receive public parameters
            BigInteger q = new BigInteger(dis.readUTF());
            BigInteger alpha = new BigInteger(dis.readUTF());
            BigInteger y = new BigInteger(dis.readUTF());

            System.out.println("Received Public Parameters:");
            System.out.println("q: " + q);
            System.out.println("alpha: " + alpha);
            System.out.println("y: " + y);

            Scanner sc = new Scanner(System.in);
            System.out.print("Enter message (integer < q): ");
            BigInteger m = new BigInteger(sc.nextLine());

            SecureRandom rand = new SecureRandom();
            BigInteger k = new BigInteger(8, rand);
```

```
            // ===== Encryption =====
            BigInteger c1 = alpha.modPow(k, q);
            BigInteger c2 = m.multiply(y.modPow(k, q)).mod(q);

            System.out.println("\nCipher Generated:");
            System.out.println("C1: " + c1);
            System.out.println("C2: " + c2);

            // Send ciphertext
            dos.writeUTF(c1.toString());
            dos.writeUTF(c2.toString());

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Receiver

```
import java.io.*;
import java.math.BigInteger;
import java.net.ServerSocket;
import java.net.Socket;
import java.security.SecureRandom;

public class elgamal_server_receiver {

    public static void main(String[] args) {

        int port = 5000;

        try (ServerSocket ss = new ServerSocket(port)) {

            System.out.println("Receiver running on port " + port);

            // ===== Key Generation =====
            SecureRandom rand = new SecureRandom();

            BigInteger q = BigInteger.probablePrime(16, rand); // small prime
for demo
            BigInteger alpha = findPrimitiveRoot(q);
            BigInteger x = new BigInteger(8, rand); // private key
            BigInteger y = alpha.modPow(x, q); // public key

            System.out.println("Public Parameters:");
```

```java
            System.out.println("Prime (q): " + q);
            System.out.println("Primitive Root (alpha): " + alpha);
            System.out.println("Public Key (y): " + y);
            System.out.println("Private Key (x): " + x);

            Socket s = ss.accept();

            DataInputStream dis = new DataInputStream(s.getInputStream());
            DataOutputStream dos = new DataOutputStream(s.getOutputStream());

            // Send public parameters to client
            dos.writeUTF(q.toString());
            dos.writeUTF(alpha.toString());
            dos.writeUTF(y.toString());

            // Receive ciphertext
            BigInteger c1 = new BigInteger(dis.readUTF());
            BigInteger c2 = new BigInteger(dis.readUTF());

            System.out.println("\nReceived Cipher:");
            System.out.println("C1: " + c1);
            System.out.println("C2: " + c2);

            // ===== Decryption =====
            BigInteger sKey = c1.modPow(x, q);
            BigInteger sInv = sKey.modInverse(q);
            BigInteger message = c2.multiply(sInv).mod(q);

            System.out.println("\nDecrypted Message: " + message);

            s.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    // Simple primitive root finder (demo version)
    static BigInteger findPrimitiveRoot(BigInteger p) {
        BigInteger phi = p.subtract(BigInteger.ONE);
        for (BigInteger g = BigInteger.TWO;
             g.compareTo(p) < 0;
             g = g.add(BigInteger.ONE)) {

            if (g.modPow(phi, p).equals(BigInteger.ONE))
                return g;
        }
        return BigInteger.TWO;
```

```
    }
}
```

Output [screenshot]

Sender

```
PS E:\aa vit sem6\z_Crypto-lab> javac elgamal_client_sender.java
PS E:\aa vit sem6\z_Crypto-lab> java elgamal_client_sender
Received Public Parameters:
q: 53051
alpha: 2
y: 3949
Enter message (integer < q): 67

Cipher Generated:
C1: 3699
C2: 2418
PS E:\aa vit sem6\z_Crypto-lab>
```

Receiver

```
PS E:\aa vit sem6\z_Crypto-lab> javac elgamal_server_receiver.java
PS E:\aa vit sem6\z_Crypto-lab> java elgamal_server_receiver
Receiver running on port 5000
Public Parameters:
Prime (q): 53051
Primitive Root (alpha): 2
Public Key (y): 3949
Private Key (x): 67

Received Cipher:
C1: 3699
C2: 2418

Decrypted Message: 67
PS E:\aa vit sem6\z_Crypto-lab>
```